

CSC3042f Assignment 2: Sequence Modelling : Encoder-decoder models for grapheme-to-phoneme conversion

Name: Timera Govender

GENAI USAGE STATEMENT

I used ChatGPT to help interpret the assignment requirements, plan the notebook structure, understand model components, and debug implementation issues. I wrote, tested, and adapted the final code myself, and verified the outputs using shape checks, training runs, evaluation tables, and plots. I also used ChatGPT to help edit the report for clarity, grammar, flow, and conciseness so that it met the assignment requirements and page limit.

1. INTRODUCTION

This assignment investigates LSTM encoder-decoder models for grapheme-to-phoneme conversion. The task maps an input word, represented as a sequence of characters, to an output sequence of phonemes. Since the input and output sequences can have different lengths, this is treated as a sequence-to-sequence learning problem. Three decoder variants are compared: a no-context bottleneck baseline, a fixed-context decoder, and an attention-based decoder.

2. DATA PREPROCESSING

The original train/validation/test splits were used without re-splitting. Character and phoneme vocabularies were built from the training set only to avoid data leakage, using fixed special tokens: <PAD>=0, <SOS>=1, <EOS>=2, and <UNK>=3. Words were lowercased and encoded as character indices, while phoneme targets were encoded with <SOS> and <EOS> added. A custom Dataset handled encoding, and a custom collate function padded variable-length batches with <PAD>, which was ignored during loss computation.

The dataset sizes were 92,426 training examples, 11,553 validation examples, and 11,554 test examples. The final character vocabulary contained 30 tokens and the phoneme vocabulary contained 73 tokens. Length histograms in the notebook showed similar word and phoneme length distributions across the three splits.

3. MODEL ARCHITECTURE

The implementation uses four main classes: a custom `LSTMCell`, an `Encoder`, a context-parameterised `Decoder`, and a `Seq2Seq` wrapper. No built-in recurrent PyTorch modules were used. The LSTM cell implements the standard forget, input, output, candidate, cell-update, and hidden-update equations manually. A context-aware version was also implemented so that the fixed-context and attention decoders could inject encoder information into each gate.

The encoder processes character embeddings one timestep at a time and returns both the full hidden-state sequence and the final hidden/cell states. The decoder generates phonemes autoregressively using teacher forcing during training and greedy decoding during inference. The three setups differ only in decoder context: no context after initialisation, a fixed final encoder hidden state at every timestep, or dot-product attention over all encoder hidden states.

4. THREE EXPERIMENTAL SETUPS

The three setups use the same encoder and `Seq2Seq` wrapper, but differ in how the decoder receives encoder information.

A. SETUP 1: NO CONTEXT/BOTTLENECK BASELINE

In the no-context setup, the encoder's final hidden and cell states initialise the decoder, but no further encoder information is provided during decoding. This forces the full input word to be compressed into a fixed-size final state, creating an information bottleneck. This setup is used as the baseline.

B. SETUP 2: FIXED CONTEXT VECTOR

In the fixed-context setup, the final encoder hidden state is passed into every decoder timestep through additional learned context-weight matrices. This gives the decoder repeated access to the encoded source representation, which reduces the bottleneck compared with Setup 1. However, the context vector is still fixed, so the decoder cannot focus on different input characters at different output positions.

C. SETUP 3: ATTENTION

In the attention setup, the decoder computes a new context vector at each timestep using dot-product attention over all encoder hidden states. The previous decoder hidden state acts as the query, attention weights are computed with a softmax over encoder states, and the context vector is formed as a weighted sum of those states. This gives the decoder flexible access to different parts of the input word while generating each phoneme, making it the strongest approach for reducing the fixed-state bottleneck.

5. TRAINING PROCEDURE

All models were trained using cross-entropy loss with the `<PAD>` index ignored, so padded positions introduced during batching did not affect the loss. The loss was computed against the target sequence excluding the leading `<SOS>` token, since `<SOS>` is only used as the decoder's starting input and is not a phoneme to be predicted.

The Adam optimiser was used for parameter updates. Because the LSTM cells were implemented from scratch, gradient clipping was applied at every optimisation step with a maximum norm of 1.0 to reduce the risk of exploding gradients.

A fixed-epoch stopping criterion was used to keep comparisons fair across model variants. Before hyperparameter tuning, each setup was trained for 5 preliminary epochs using the same configuration: embedding dimension 64, hidden size 128, one LSTM layer, learning rate 0.0001, and batch size 64. In all three setups, both training and validation loss decreased consistently, confirming that the training loop, loss calculation, optimiser, and gradient clipping were working correctly. The final architecture comparison later used the same fixed-epoch approach, but with the selected tuned hyperparameters and a larger 10-epoch training budget.

6. HYPERPARAMETER TUNING

Hyperparameter tuning was performed using the no-context baseline model, since it is the simplest setup and gives a consistent basis for comparing configurations. A greedy tuning strategy was used: learning rate was tuned first, followed by embedding dimension, and then hidden size.

Each configuration was trained for 5 epochs. Validation loss was computed on the full validation set, while validation PER was estimated using 30 validation batches to reduce runtime. PER was used as the main selection metric because it directly measures phoneme prediction quality.

Hyperparameter	Values tested
Learning rate	0.001, 0.0001, 0.00001
Embedding dimension	32, 64, 128
Hidden size	64, 128, 256
Number of LSTM layers	1

Table 1 Hyperparameters tested during validation-based tuning using the no-context baseline model.

The best configuration was:

Learning rate	Embedding dimension	Hidden size	Layers	Validation PER
0.001	128	256	1	0.1376

Table 2 Best hyperparameter configuration selected from validation PER using the no-context baseline model.

The learning rate of 0.001 performed best because it allowed the model to learn effectively within the short 5-epoch tuning budget, while smaller learning rates converged too slowly. Increasing both the embedding dimension and hidden size improved validation PER, suggesting that the model benefited from richer character embeddings and greater recurrent capacity. This selected configuration was reused for all three final setups so that performance differences could be attributed to the decoder architecture rather than different hyperparameters.

7. ARCHITECTURE COMPARISON AND ANALYSIS

The three decoder setups were trained under identical conditions using the selected hyperparameters: learning rate 0.001, embedding dimension 128, hidden size 256, one LSTM layer, batch size 64, the same random seed, and a fixed 10-epoch training budget. This ensured that performance differences were due to the decoder architecture rather than the training configuration. The training and validation loss curves for all three setups follows:

The convergence summary is shown below:

Setup	Initial Validation Loss	Final Validation Loss	Best Validation Loss	Best Validation Epoch
No Context	0.5342	0.2935	0.2932	9
Fixed Context	0.5291	0.2930	0.2871	8
Attention	0.4615	0.2603	0.2603	10

Table 3 Validation-loss convergence summary for the three final decoder setups trained under identical conditions.

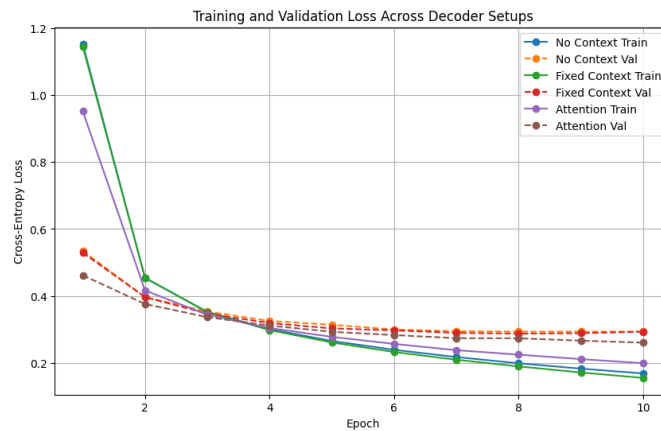


Figure 1 Training and validation cross-entropy loss curves for the no-context, fixed-context, and attention decoder setups trained under identical conditions.

The training and validation loss curves show that all three models learned successfully, with loss decreasing rapidly in the first few epochs before improving more slowly. The fixed-context model reached its best validation loss earliest, at epoch 8, while the no-context model reached its best value at epoch 9. However, the attention model achieved the lowest validation loss overall, reaching 0.2603 at epoch 10, which suggests it learned more effectively and may have benefited from further training.

The final test results are shown below:

Setup	Test Loss	Test PER	Word Accuracy	Word Accuracy (%)
No Context	0.3100	0.1302	0.5660	56.60%
Fixed Context	0.3075	0.1290	0.5653	56.53%
Attention	0.2742	0.1237	0.5685	56.85%

Table 4 Final test-set performance comparison for the three decoder setups using test loss, phoneme error rate, and word accuracy.

The attention model achieves the lowest test loss, with a test loss of 0.2742. It also achieves the lowest test PER and the highest word accuracy. This indicates that the attention-based decoder provides the strongest overall performance among the three setups.

The fixed-context model gave only a small improvement over the no-context bottleneck baseline. Test loss decreased by 0.0025 and PER decreased by 0.0012, although word accuracy decreased slightly from 56.60% to 56.53%. This suggests that repeatedly passing the final encoder hidden state to the decoder reduces the bottleneck slightly, but still limits the model to a single compressed representation of the input word.

The attention model improved more clearly. Compared with the no-context model, attention reduced test loss by 0.0358, reduced PER by 0.0065, and improved word accuracy by 0.25 percentage points. Compared with fixed context, attention reduced test loss by 0.0333 and PER by 0.0053. This suggests that dynamic context is more useful than a single fixed context vector, because different output phonemes may depend on different input character positions.

Error analysis supports this interpretation. For vulgarization and dickensheets, the no-context model produced incorrect vowel phonemes, while the attention model generated the correct sequences. Both are relatively long words, suggesting that the bottleneck model can lose useful character-level information as sequences become longer. For lucca, the no-context model inserted an extra final phoneme, while attention correctly stopped after the final phoneme. These examples suggest that attention helps both with local character-to-phoneme alignment and with deciding when to end generation.

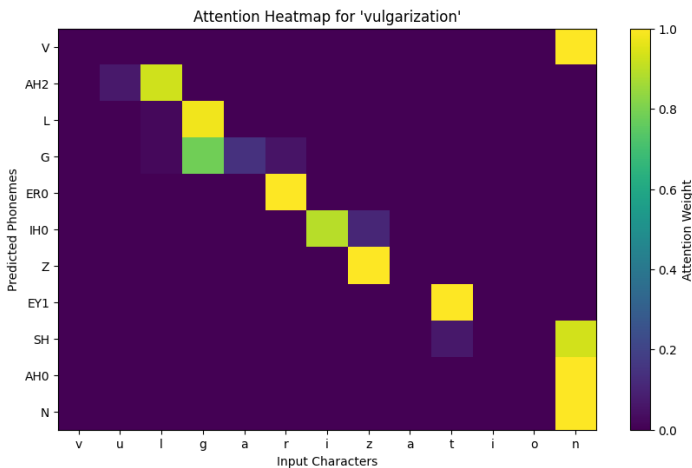


Figure 2 Attention heatmap for the word vulgarization, showing how the attention decoder distributes focus over input characters while generating each predicted phoneme.



Figure 3 Attention heatmap for the word dickensheets, showing how the attention decoder aligns predicted phonemes with relevant input characters during generation

Attention heatmaps for vulgarization and dickensheets show mostly diagonal patterns, indicating that the attention decoder learned a soft alignment between input characters and output phonemes. Later output phonemes attend to later input characters, which supports the quantitative results: attention improves performance by allowing the decoder to refer back to relevant encoder states instead of relying only on the final encoder state.

8 EVALUATION

The final models were evaluated on the test set using phoneme error rate (PER) and word accuracy. The test set was only used after training and hyperparameter selection were complete.

Phoneme error rate measures how different a predicted phoneme sequence is from the reference phoneme sequence. For each test example, the edit distance between the predicted and reference phoneme sequences was computed. This edit distance counts the minimum number of phoneme insertions,

deletions, and substitutions required to transform the prediction into the reference. The PER for a single word was then calculated as:

$$PER = \frac{\text{edit distance}}{\text{length of reference phoneme sequence}}$$

The reported PER is the average PER across all test examples. Lower PER indicates better phoneme-level performance.

Word accuracy was also computed. This is the fraction of test words for which the entire predicted phoneme sequence exactly matched the reference sequence. Word accuracy is stricter than PER because a prediction is counted as incorrect if even one phoneme differs from the reference.

Before computing these metrics, predicted and target token sequences were converted back into phoneme symbols. The <PAD> and <SOS> tokens were removed, and decoding stopped when <EOS> was reached.

The final test results are shown below:

Setup	Test Loss	Test PER	Word Accuracy	Word Accuracy (%)
Setup 1: No Context	0.3100	0.1302	0.5660	56.60%
Setup 2: Fixed Context	0.3075	0.1290	0.5653	56.53%
Setup 3: Attention	0.2742	0.1237	0.5685	56.85%

Table 5 Final test-set evaluation results for all three decoder setups, comparing test loss, phoneme error rate, and word-level accuracy.

The attention-based model achieved the best overall evaluation results. It had the lowest test loss, the lowest PER, and the highest word accuracy. The fixed-context model slightly improved over the no-context model in terms of test loss and PER, but its word accuracy was slightly lower. This suggests that the fixed context vector provided only a small benefit, while attention provided a more meaningful improvement by allowing the decoder to use different encoder hidden states at different decoding steps.

9 (A) LENGTH ANALYSIS

To analyse whether model performance changes with input length, the test words were grouped into four word-length buckets: 1–4, 5–7, 8–10, and 11+ characters. For each bucket, word accuracy was computed separately for the no-context, fixed-context, and attention-based models. The results are shown below.

Word Length Bucket	No Context	Fixed Context	Attention
1–4	0.7701	0.7602	0.7491
5–7	0.6291	0.6354	0.6230
8–10	0.4887	0.4825	0.4983
11+	0.3453	0.3317	0.3872

Table 6 Word accuracy by input word-length bucket for each decoder setup on the test set.

Word accuracy decreased for all three setups as word length increased. This is expected because longer words usually require longer phoneme sequences, which creates more opportunities for at least one phoneme error. Since word accuracy requires the entire predicted phoneme sequence to match the reference exactly, even a single substitution, insertion, or deletion causes the whole word to be counted as incorrect.

For short words of length 1–4, the no-context model achieved the highest word accuracy, with 0.7701. The fixed-context and attention models performed slightly worse in this bucket. This suggests that for very short inputs, the bottleneck is less harmful because the full word can be compressed into the encoder’s final state more easily.

However, for the longest words in the 11+ bucket, the attention model performed best. It achieved a word accuracy of 0.3872, compared with 0.3453 for the no-context model and 0.3317 for the fixed-context model. This supports the idea that attention is more useful for longer input sequences, where compressing the entire word into a single final encoder state is more likely to lose useful character-level information.

The no-context bottleneck model degraded substantially as word length increased, falling from 0.7701 on 1–4 character words to 0.3453 on 11+ character words. However, all models showed degradation with increasing length. The main difference is that attention performed best on the longest words, suggesting that dynamic access to encoder hidden states helps the decoder preserve and use relevant source information across longer sequences.

Overall, the length analysis supports the bottleneck hypothesis. The no-context and fixed-context models rely on a single compressed source representation, while the attention model can refer back to different encoder hidden states during decoding. This appears to be especially beneficial for longer words, where detailed character-to-phoneme alignment is harder to maintain.

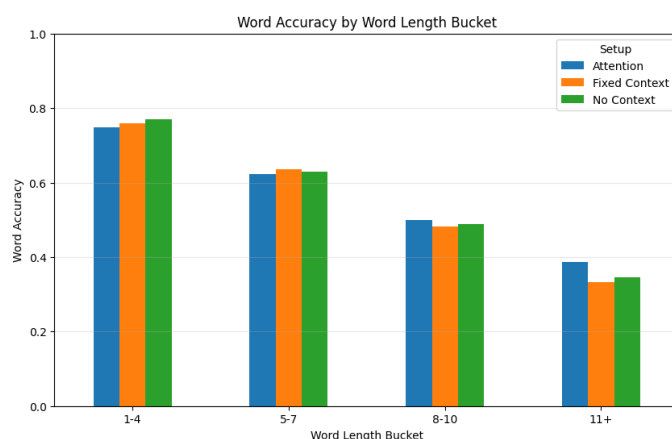


Figure 4 Grouped bar chart showing word accuracy across input word-length buckets for the no-context, fixed-context, and attention decoder setups on the test set.